

Overview

StudyCAT is a computer-adaptive practice tool for large university classes. Our primary client is Prof. Jason De Melo, who teaches courses with 500–1000 students where midterms are MCQ-based and personalized practice is hard to provide. Students will sign in with UTORid, choose a concept-focused practice or a pre-midterm set, and work at their own pace. Quizzical is already in use for question authoring and identifying proficient students, and a tagged question repository is expected by late November. StudyCAT complements this by adding adaptive practice and clear feedback. The target is a student-ready tool by September 2026.

The system picks each next MCQ based on recent performance and estimated mastery across multiple concepts and question types using an IRT (Item Response Theory)-based engine. Students can stop at any time of their choosing or when a mastery plateau is reached. Progress auto-saves and feedback appears as a spiderweb plot that shows strengths and gaps by domain. Personalization uses tags to narrow the bank and to focus on weak areas. Each item stores difficulty, discrimination, and a randomness factor. Instructors can tie questions to concepts, reweight what matters for a midterm, organize quizzes by lecture, remove or swap items, adjust time or count, review question and student performance, see how many questions a student needed to reach a plateau, and export engagement and performance data by student identifier. The design is extendable across semesters and courses.

This semester's MVP is one continuous adaptive quiz, useful student feedback, and a basic instructor view. Automated question labeling and summative assessment are out of scope for now. The stack is a Next.js web app with serverless APIs, a Python modeling service, and PostgreSQL with Prisma. We plan a prototype by early December 2025. The approach is grounded in education research and will likely reuse an open-source IRT library to reduce implementation risk.

Context

The idea for StudyCAT was born from a need for a more personalized practice testing experience in contexts where resources are limited. In large classes of thousands of students, it is near impossible to provide each student with meaningful feedback and practice before a formative assessment. A system which could tailor practice test questions to each individual student according to their strengths and weaknesses, as well as provide instructors with insights about where their class was struggling, would be ideal to mitigate these issues and reduce strain on professors, TAs, and students.

A computerized adaptive learning program such as StudyCAT will provide large benefits to both students and instructors alike. Students can prepare for formative assessments in a variety of different ways, but one of the most effective ways for students to prepare is by doing practice tests. Practice tests force students to retrieve information and apply it directly to questions that are similar to the types of questions they would see on the real exam, which strengthens that

information in memory (Oakley & Sejnowski, 2018). However, one difficulty with pre-made practice tests is that they clearly cannot be tailored to individual students. Students that have mastered certain concepts and struggle with others may not benefit as much as they could from a traditional practice test because that test might not include enough questions about topics they are struggling with. StudyCAT will attempt to fill this gap, by recognizing where students are struggling and focusing on giving them questions that are related to the topics they need more practice on and adjusting the difficulty level of the questions they are getting relative to their current skills.

Traditional practice tests provide some value to instructors, but with more detailed data, instructors would be able to improve their teaching styles and classes greatly. When an instructor provides their students with a traditional practice test, it is somewhat difficult for them to gather feedback about the ways that students are using these tests and where students are finding difficulties. Unless the instructor asks students to directly provide feedback, there is little insight that can be gained from providing students with a practice test. With StudyCAT, instructors will gain access to detailed statistics about student performance, giving them a much better idea of where exactly students are struggling, so they can provide added support.

Existing Solutions

There are several existing solutions that allow for the custom implementation of computerized adaptive testing (CAT). One category consists of industry-focused platforms such as FastTest or Questionmark, which are widely used for certification and hiring. These services provide features such as IRT-based MCQ banks, robust analytics, and professional user interfaces. However, they are typically expensive, require substantial psychometric expertise, and are not designed with pedagogy or formative learning in mind.

A second option is the Concerto Platform, developed at the University of Cambridge. Concerto is a self-hosted, open-source system that enables IRT-based CAT quizzes to be delivered through a web interface. It is flexible, research-driven, and capable of generating detailed feedback reports. Nevertheless, its reliance on technical setup and maintenance presents a significant barrier for instructors. For a professor aiming to create adaptive quizzes on a regular basis, the workload and infrastructure requirements may prove prohibitive.

Finally, adaptive testing functionality can be found in existing learning management systems (LMS). For instance, Moodle supports adaptive quiz plugins that adjust question difficulty based on student performance. While accessible, these plugins typically implement only simplified forms of adaptivity rather than true CAT. Their features and reliability vary across implementations, and they require the use of the broader Moodle ecosystem, which may not be feasible or desirable in this context.

At the University of Toronto, the Quizzical tool represents another relevant solution, enabling large-scale student authorship of MCQs, allowing instructors to collect and curate question

banks from their classes. While not itself adaptive, this functionality is critical for building the item pool that will support the development of StudyCAT.

In conclusion, while existing solutions provide valuable insights into how CATs can be delivered, they either prioritize high-stakes assessment, impose technical burdens, or offer only partial adaptivity. None are ideally suited for Professor de Melo's use case, where the goal is to provide accessible, pedagogically-driven adaptive quizzes that improve student understanding. StudyCAT is designed to fill this gap by combining ease of use for instructors with true adaptivity for students, all within the context of undergraduate learning.

Goals

- Deliver prototype by Dec 2, 2025, and a fully usable tool for students by Sept 2026.
- Login page connected to students' UTORids.
- Option to take a test on a particular concept or a pre-midterm test.
- Each next question is chosen based on student performance.
- Personalized feedback:
 - Allow students to stop anytime and receive feedback.
 - Show mastery progress with spiderweb plots across different domains and question types.
 - Progress saves automatically and updates mastery over time.
- Extendable to multiple semesters and courses
- Instructor insights:
 - View student performance and mastery plateaus.
 - Organize quizzes by lecture/topic.
 - Adjust parameters (e.g., time limits, number of questions).
- Backend data tracking:
 - Record question attempts and timestamps.
 - Export data linked to student identifiers for instructors (e.g., participation grades).

MVP Goals

- One infinite quiz
- Next questions being chosen based on student performance
- Provide students with 'useful' feedback (how can we measure this?)

Non-goals

- Does not classify or label questions automatically. Needs pre-labelled data.
- Not an official exam/assessment platform (at least initially) – positioned as a practice tool first.

- Not focused on automatic question generation – relies on instructor- or student-written questions.

Proposed Solution

Primary flows

Student

1. Sign in with UTORid, land on Dashboard to select course.
2. Start new session: pick Concept Practice or Pre-Midterm, choose scope by lectures, concepts, and question types.
3. Answer one MCQ at a time. The next item is chosen based on performance and current mastery.
4. End session when manually selected or when mastery is attained.
5. See results: spiderweb summary of strengths and gaps, per-item feedback, “practice next” suggestions. History and auto-resume are available.

Instructor

1. Sign in and select course or term.
2. View cohort overview of concept mastery, engagement over time, and items to plateau.
3. Reweight concepts for an upcoming midterm and organize practice by lecture.
4. Review student and item analytics, remove or swap items, adjust time limits or item counts.
5. Export CSV keyed to student identifier for participation credit.

Our design of the primary flow has been informed by discussion with the primary client, as well as research about helpful features of adaptive testing systems. Nursing students using an AQS system indicated that some of the most useful features were the convenience of the web-based design, the ability to track their progress throughout the year, an easy-to-use interface, and the connection to course content (Malkemes & Phelan, 2017). Therefore, we aim to implement all of these key features in order to maximize the effectiveness of our system for all students.

Top user stories

Student

- I can start practice within three clicks and see my scope before I begin.
- I get progressively more difficult questions if I am acing through the easy ones.
- I can stop any time and get clear feedback with a spiderweb plot and next steps.
- My progress auto-saves and I can resume past sessions.

Instructor

- I can see where the class struggles by concept and question type.
- I can set weights that shape the default Pre-Midterm scope for students.

- I can see when the class attempts which question.
- I can audit or retire weak items and export clean participation data.

Model Details

Our model will be based on Item Response Theory, with a 3-PL structure. 3-PL models include 3 parameters for questions: difficulty level, discriminative power of a question, and the likelihood of guessing. We have chosen this structure due to the nature of StudyCAT's multiple choice questions. There is a consistent ~25% chance of guessing the correct answer for each question, which should be considered in the estimation of a student's ability. We also considered using a 4-PL structure, which includes all parameters of a 3-PL model along with an 'upper asymptote' parameter which accounts for the probability that a high-ability student might get a question wrong for some reason unrelated to ability, such as carelessness, misclick, or trick wording. However, we felt this fourth parameter was somewhat unnecessary, especially because in the absence of a large calibration phase, someone will have to manually estimate this chance.

The decision to create a CAT system with an IRT approach has been informed by literature, as well as discussion with our primary client. Our primary client has expressed an interest in a 3-PL IRT model, as was implied from his discussion of what characteristics each question should include (score, difficulty rating, and randomness factor). A CAT system with an IRT approach has been shown to be considered highly practical and useful by students (Almasri et al., 2024). Nursing students using an ACQ, something similar to a CAT system, saw an 11.55% increase in pass rates for the NCLEX-RN test (Malkemes & Phelan, 2017). Overall, a CAT system with an IRT approach seems to make the most practical sense for our purposes.

Because we intend to assess students' mastery of multiple different concepts and question types, we believe a multidimensional IRT model is necessary. Multidimensional IRT models keep track of several 'ability' values per student, which all correspond to different skills/concepts in the course. Each question will need to be tagged with the concept it covers, as well as the other characteristics mentioned previously. In the question selection process, concepts must be prioritized/deprioritized based on not only student performance, but also instructor input. In addition, we have given thought to the idea of having 'ability' values not only for concepts, but also question types (recall, application, etc.). For example, instead of a broad 'stoichiometry' ability value, we might have multiple different values: 'stoichiometry recall', 'stoichiometry application', etc. This may allow us to more accurately estimate how well a student truly understands a specific topic, and would give them even more personalized feedback about what they need to work on.

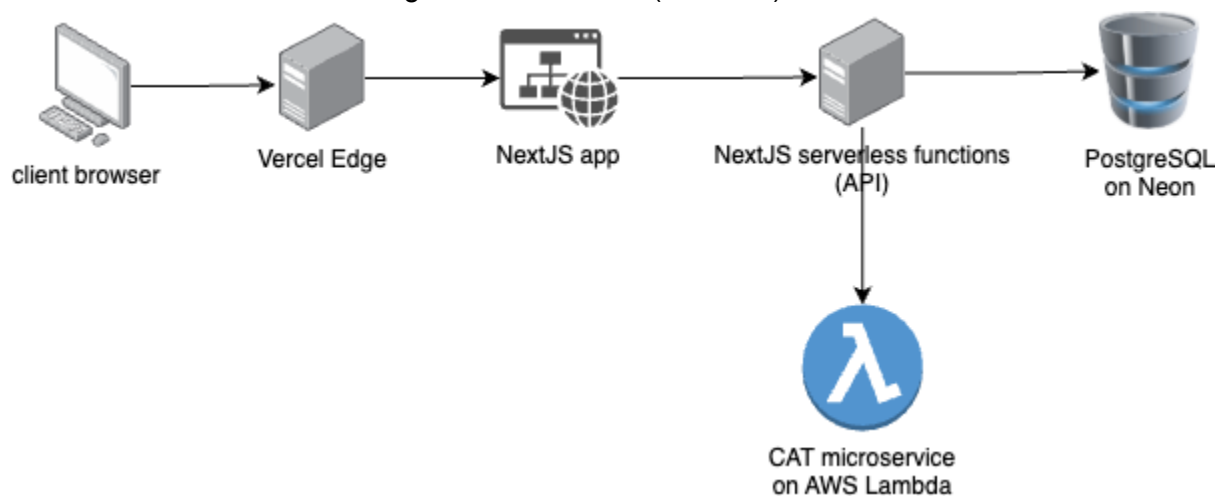
Another factor that must be considered is so-called 'mastery levels' of concepts. There should be a kind of 'mastery threshold', where we can reasonably assume a student does not need to continue to see questions of the same type because they have effectively mastered that specific topic. Real-world research will need to be conducted in order to determine exactly where this threshold should land. Thresholds may need to be customizable, not only based on course, but per concept as well. It has also been shown that review of previously mastered concepts can be

helpful for students (Kang et al., 2020), therefore it might be beneficial to not completely abandon mastered concepts, but instead deprioritize them enough to allow time for other concepts, while still sprinkling in mastered concepts from time to time.

We have decided to create our model in Python, both due to the abundant machine learning resources available within Python as well as the team's personal familiarity and comfort level with the language. In addition, there are several existing Python libraries which demonstrate the viability of an IRT model created in Python, such as 'catsim' (<https://github.com/douglasrizzo/catsim?tab=readme-ov-file>). The 'mirt' package in the R language also might be a helpful resource for us, as it implements multidimensional IRT models.

Tech Stack

- [Next.js](#) with serverless functions for API
 - Hosted on Vercel (tentative)
- Python for CAT model
 - Packaged as an API and deployed as a serverless function (AWS Lambda) (tentative)
- Prisma ORM + PostgreSQL
 - Serverless PostgreSQL with Neon (tentative)



Milestones

Milestone 1: Foundations and research (week 1, 2)

- Finalize requirements and research
 - Confirm MVP scope (infinite quiz, adaptive question selection, basic feedback)
 - Document question metadata needed for CAT (difficulty, discrimination, etc.)
- CAT algorithms research
 - Decide on initial adaptive algorithm

- Identify any data requirements and approaches for calibration

Milestone 2: Designs (weeks 3, 4)

- UI/UX prototypes in Figma
 - Student flow: login → start quiz → adaptive questions → feedback screen
 - Instructor flow: basic quiz setup, viewing responses (low-fidelity for now)
- Feedback loop
 - Get input from professors and iterate on designs

Milestone 3: Technical setup (weeks 5, 6)

- Backend infrastructure
 - Set up UTORid mock login (real SSO later).
 - Define database schema and setup development database
 - Setup basic API routes
- Frontend skeleton
 - Create NextJS app with pages based on designs

Milestone 4: Core CAT development (weeks 7, 8, 9)

- Quiz engine (infinite quiz)
 - Implement random baseline quiz loop
 - Adaptive algorithm integration
 - Connect quiz engine to algorithm to choose next question.
- Implement feedback for students and instructors
- Validate with testing using dummy data

Milestone 5: Deployment and testing (weeks 10, 11)

- Basic deployment
- Ensure demo readiness
- Outline future roadmap
- Validate product with testing

Questions

- How can we measure the success of this product?
- What will the labelled question bank data look like? When can we get a small sample so we can maybe generate fake data to start building the model structure?
- How do we know if a model works well versus another?
- Should we have different theta values (ability values) for each different type of question (recall, application, etc.) and concept?

References

Almasri, Firdaus, Hendriyani Y., Huda A., Irfan D., Sukmawati M. (2024). Optimizing Educational Assessment: The Practicality of Computer Adaptive Testing (CAT) with an Item Response Theory (IRT) Approach. International Journal on Informatics Visualization. <http://dx.doi.org/10.62527/ijiv.8.1.2217>

Kang S., Rennie O., Riggs D. (2020). Positive Impact of Multiple-Choice Question Authoring and Regular Quiz Participation on Student Learning. DOI:10.1187/cbe.19-09-0189

Malkemes, S. and Phelan, J.C. (2017) Impact of Adaptive Quizzing as a Practice and Remediation Strategy to Prepare for the NCLEX-RN. Open Journal of Nursing, 7, 1289-1306

Oakley, B., & Sejnowski, T. (2018). Learning how to learn. Penguin.